



Darren Waine

**VGP241**

Algorithms and Design Patterns

# Agenda

- Hash Tables
- Sorting
- Bubble Sort
- Select Sort

# Hash Tables

- What are hash tables?
- How are they created?
- Why they exist in computer science algorithms?

# What Are Hash Tables

- Fast data structure
- Based on arrays
- Used to hold a database (indexed data) of information in memory

# What Are Hash Tables

- Speed comes from the fact the hash tables are based on array data structures
- When the array index of an object is known, the position is just a table lookup using the index
- The trick is calculating the object's array index position

# What Are Hash Tables

- Since they are based on arrays, they have their strengths as well as their weaknesses
- When the size of the hash table can be predicted, some weakness may not be an issue

# Hash Table Facts

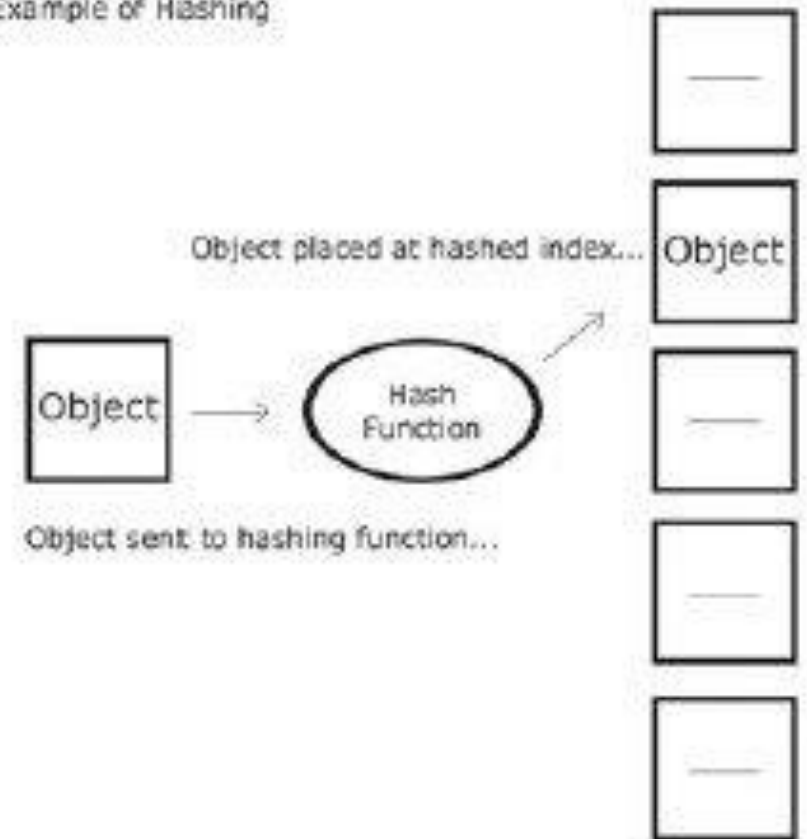
- Are expensive and hard to expand
- Are further complicated because the positions of the objects within the table depend on the size of the data structure
- If a change in size, objects added are then invalidated
- When using open addressing, the performance can decrease as the container becomes full
- Objects cannot be ordered easily within the hash table
- Copying can be expensive and requires that both tables be the same size to avoid having to rehash all existing elements
- Lookup objects is fast and uses array subscripts
- Insertions are fast

# How Are They Created

---

- Work by taking an object and hashing it to a number
- This number is used as an array position for that object
- "Hashing" a value refers to executing an algorithm on the object to create an integer that lies within the array bounds of the hash table
- Every object that is inserted has its own key
- Key is an identifier that is hashed into an array index and used for the object
- The key can be anything, even the object itself
- The code used to hash an object into an array index is called a "HashFunction"
- With large datasets, the speed that hash tables operate is impressive

Example of Hashing





# Hash Table Uses

- Spell checkers
- Database systems
- Dictionaries
- Phone directories
- Implementation of a symbol table for a compiler or scripting system
- Implementation of a caching system
- Implementation of a transposition table in a chess game
- List of players in a MMORPG
- Storing player's stats, items, weapons, equipment, character information...

# Performance

- Speed is not dependent on the number of objects
- Means hash tables with 1,000,000 elements would be as fast as hash tables with 100 elements
- Time complexity is close to  $O(1)$  in most operations
  - Main dependent is number of collisions

# Hash Function

- Is an algorithm that takes a value and compresses it into the range of the hash table's array
- Value is known as the "key"
- Key can be anything from a string, number, address...
- Hash value of the key depends on the size of the hash table's array
- If the size changes, the added objects would all have to be rehashed and repositioned to new elements
- Efficiency is very important
- Poor function can lead to collisions in the data structure
- A collision occurs when more than one object hashes to the same index
- There are methods to do when collisions occur
- No hash function can guarantee that there will be no collisions, though when they occur, they must be handled

# Hashing Values

- When hashing a key that is a number, we can use a simple equation to demonstrate how to create a hash function
- The has function will be based on the modulus operator (%) and will be used in conjunction with the array size to return a value within the array bounds
- Eg:  $\text{key} \% \text{size} = \text{hash value}$

# Key Points

- Hashing needs to be:
  - Stable
  - Uniform
  - Efficient
  - Secure

# Hashing A String (First Try)

- Naïve implementation would be summing the ASCII values of each character
- Eg: hash for "foo" would be  $102 + 111 + 111 = 324$

```
int HashFunction(string str)
```

```
    return hashCode
```

- Once we have the hash, we can get an index for our item
- In our implementation we can return the index for the item from the HashFunction
- We calculate the array index from the has by
  - $\text{arrayIndex} = \text{hashCode} \% \text{arraySize}$
- Once we have the index, we save the object to the position in the array
  - `mArray[arrayIndex] = data;`

# Hashing A String (Second Try)

- In this example, we'll do the same, but add up all the characters of the string and do some arithmetic to build the hash value
- When looping through the key, we can use both the current hash value, the total number of letters that can be used, and the value of the letter itself along with the modulus operator to create the hash value
- Small strings can be hashed this way

```
for(int i = 0; i < (int)str.size(); ++i)
```

```
    Int val = (int)str[i];
```

```
    Hash = (hash * 256 + val) % size;
```

# Working With Hash Tables

- First have to find a position in the hash table that corresponds to an item
- If we are inserting and there is something already inserted in the array index, a collision will occur
- If this happens, a new position must be found
- Even if not inserting, but searching, if the item is not in the initial hashed index, we need to find it elsewhere
- These are known as resolving collision



# Implementing Hash Tables

- The key is an integer that is hashed by the has table, while the object can be anything
- The member functions of the class include functions for getting and setting the key as well as the object, adding, finding, and growing the array
- There should also be a bool comparison function for comparing two hash items
  - `bool operator==(const MyClass& other)`

# Exercise

- Create a class called Texture
- Have an array of floats for pixel colors
- Create it with size, initialize array with  $\text{size} * \text{size} * 4$ 
  - Width, Height, and RGBA
- Create a hash table array that will store the textures
- Create 100 of textures and add them to the hash table, using their name as the key (store the hash value)
- Iterate through the hash values and print the name of each texture

# Sorting

- Algorithms used rearrange items in a container in a specified order
- There are different algorithms for different types of data
- Algorithms may differ depending on the output requirements
- General purpose sorting methods are available in most programming languages (eg: `std::sort()`)

[Sorting Algorithms - GeeksforGeeks](#)

# Sorting Advantages

- Can make data easier to find
- Can improve performance based on the order of the data
- Simplified data analysis, easier to identify patterns
- Easier to visual data in graphs and charts

# Sorting Disadvantages

- Insertion operation can be costly as the order generally has to be maintained
- If its not maintained, then the sort algorithm can be costly when called
- Algorithm choices depend on the type of data sorted
- Hashing can be better than sorting in some cases

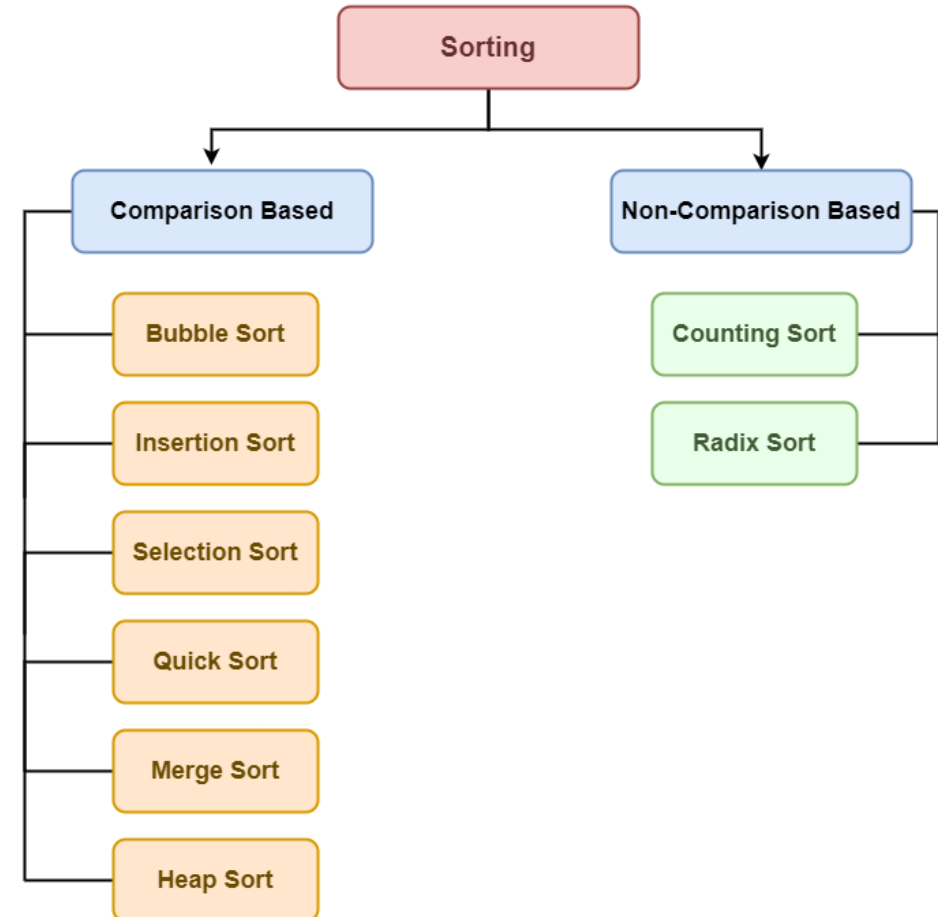
# Different Types Of Sorting

- In-Place Sorting
  - Uses constant space for producing output, just modifies given array
  - Bubble Sort\*, Selection Sort\*, Heap Sort, Insertion Sort
- Internal Sorting
  - All data is in the main memory and input cannot go beyond allocated memory size
- External Sorting
  - All data needs to be sorted, but not need to be placed in memory at a time
  - Used for massive amounts of data, whole array doesn't need to stay in memory the whole time (eg: Merge Sort)
- Stable Sorting
  - When two items appear in the same sort
- Hybrid Sorting
  - Uses more than one form of sorting algorithms
  - Intro Sort

# Types Of Techniques

---

- Comparison Based
  - Compare the elements in the container
- Non-Comparison Based
  - Elements in the container are not compared



# Sorting Algorithms

- Bubble Sort –  $O(n^2)$  Time and  $O(1)$  Space
  - Swaps adjacent values until all sorted
- Insertion Sort –  $O(n^2)$  Time and  $O(1)$  Space
  - Sorting one item at a time
- Selection Sort –  $O(n^2)$  Time and  $O(1)$  Space
  - Finds the smallest/largest elements and swaps with it
- Counting Sort
  - Counts frequency of value and puts in index in sorted positions



# Bubble Sort

- Simplest sorting algorithm
- Repeatedly swaps adjacent elements if they are in the wrong order
- Repeats passes until everything is sorted
- Compare first two elements
- Swap if they are not in order
- Advance one and repeat
- Largest item will be at the end, so ignore it next pass

[Bubble Sort in C++ - GeeksforGeeks](#)

# Exercise

- Create a vector with 100 random numbers
  - Print numbers
- Use a Bubble Sort to sort them from lowest to highest
  - Print numbers

# Selection Sort

- Repeatedly finds the lowest (or highest) element in the container
- The found element is then swapped with the beginning of the unsorted set
- Maintains two sub arrays in a given array
  - The sub array that is already sorted
  - The sub array that is unsorted
- Every iteration moves the smallest (or highest/target) to the beginning of the list

[C++ Program For Selection Sort - GeeksforGeeks](#)

# Exercise

- Create a vector with 100 random numbers
  - Print numbers
- Use a Selection Sort to sort them from lowest to highest
  - Print numbers